Satellite Land Cover Classifier

Requirements Gathering

Stakeholder Analysis • User Stories & Use Cases • Functional & Non-Functional Requirements

EuroSAT 13-Band Deep Learning Pipeline + Streamlit Inference App

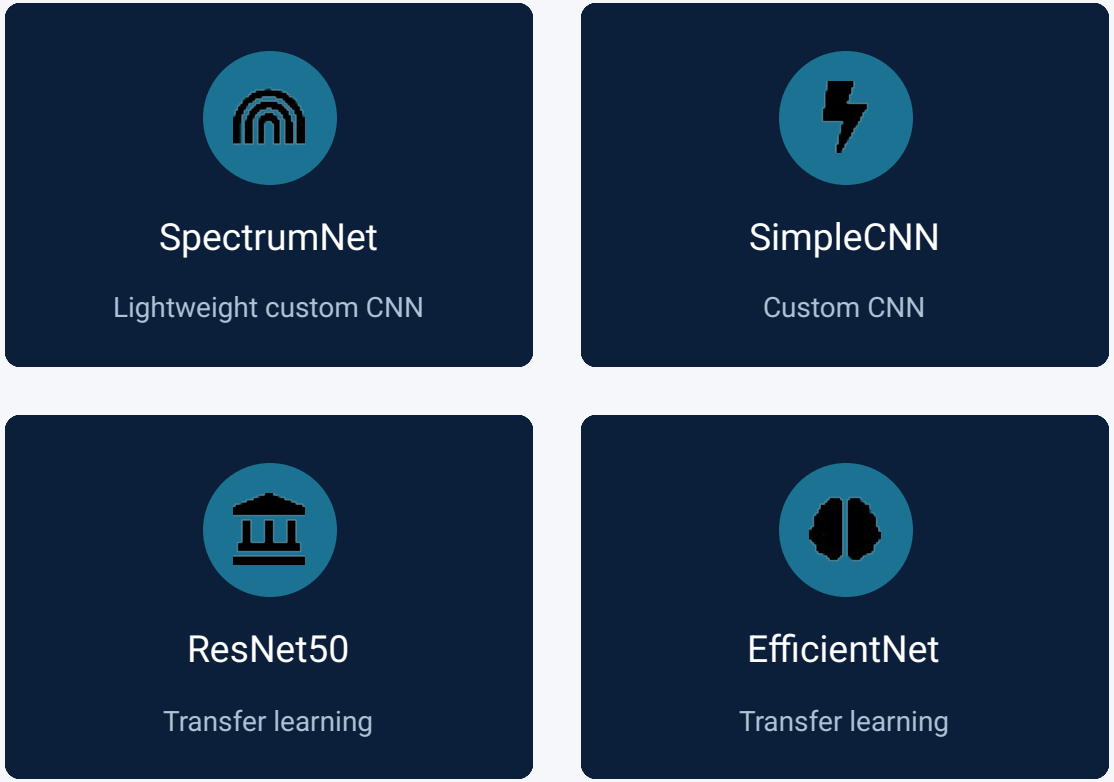
Project Overview

What the system does, end to end

A deep-learning pipeline trains and compares four CNN architectures on the EuroSAT 13-band Sentinel-2 dataset, classifying scenes into 10 land-cover classes.

A companion Streamlit web app lets a user upload an image, choose a model, and get an instant classification with confidence scores.

- Handles both real 13-band satellite imagery and ordinary RGB photos
- 10 land-cover classes, from Forest and River to Residential and Highway
- Model registry lets users trade off speed vs. accuracy



Stakeholder Analysis

Who this system serves, and what each group needs from it

End User / Analyst

Simple upload flow, clear results, and confidence scores they can trust.

Data Scientist

Clean pipeline and comparable metrics across all four models.

App Maintainer

Clear model registry so new models drop in without UI rewrites.

Domain Expert

Reliable predictions on real imagery for real-world decisions.

Instructor / Evaluator

A working end-to-end demo with documented model comparisons.

Casual / Non-Technical User

Still gets a result from an ordinary photo, with honest caveats.

User Stories

What each stakeholder wants to accomplish, in their own words (1/2)

END USER

Upload a satellite image and immediately see the predicted land-cover class.

END USER

Switch between lightweight and transfer-learning models to trade speed for accuracy.

END USER

See a confidence score and full class-probability breakdown before trusting a result.

CASUAL USER

Upload a regular photo, not just a real 13-band file, and still get a result.

CASUAL USER

Be warned clearly when a prediction is only an approximation, not real satellite data.

User Stories

What each stakeholder wants to accomplish, in their own words (2/2)

DATA SCIENTIST

Train and evaluate all four models on the exact same data split for a fair comparison.

DATA SCIENTIST

Have each model's Accuracy, Precision, Recall and F1-score recorded to justify a recommendation.

APP MAINTAINER

Keep model metadata centralized in one registry, so adding a model doesn't touch UI logic.

APP MAINTAINER

Have models loaded lazily and cached, so switching models in the UI stays fast.

Together, these 9 stories span all six stakeholder groups — from the end user running a single prediction to the maintainer keeping the model registry clean.

Use Cases — Core Prediction Flows

How the app behaves for real satellite data vs. ordinary photos

UC-1: Classify a Real Satellite Image

13-band .tif upload

1. User selects a model (e.g., EfficientNet).
2. Uploads a 13-band .tif file.
3. System validates bands, resizes, previews true-color RGB.
4. User clicks Run Prediction.
5. System shows class, confidence, and probabilities.

UC-2: Classify an Ordinary Photo

jpg / png / bmp upload

1. User uploads a standard RGB photo instead of a .tif.
2. System fabricates a synthetic 13-band array from RGB.
3. System displays a clear approximation warning.
4. User may download the synthetic image as .tif.
5. User runs prediction, aware of reduced reliability.

Use Cases — Model Comparison & Error Handling

Supporting workflows behind the scenes

UC-3: Compare Models

Notebook training & evaluation

1. Data scientist trains all four architectures on the same split.
2. Each model's Accuracy, Precision, Recall & F1 are computed.
3. A comparison table guides the app's default model choice.

Outcome: an evidence-based recommendation for which model(s) the app should default to.

UC-4: Handle Invalid Input

Robust failure handling

1. User uploads a .tif without exactly 13 bands, or a corrupt file.
2. System catches the error before it reaches the model.
3. A readable message is shown instead of a crash.

Outcome: the app stays stable and understandable, even on bad or unexpected input.

Functional Requirements

Complete list — FR-1 through FR-8 (data & model pipeline)

FR-1 Load and preprocess 13-band EuroSAT imagery for training (resize, label & one-hot encode).

FR-2 Split data into stratified 80/10/10 train / validation / test sets.

FR-3 Train four architectures — SpectrumNet, SimpleCNN, ResNet50 & EfficientNet — on a (64,64,13) input.

FR-4 Evaluate each trained model with Accuracy, Precision, Recall and F1-score on the held-out test set.

FR-5 Persist each trained model as a .h5 file for later inference.

FR-6 Let the user select which of the four models to use for prediction.

FR-7 Accept .tif/.tiff uploads and validate that they contain exactly 13 bands.

FR-8 Accept jpg/jpeg/png/bmp uploads and construct a synthetic 13-band array from the RGB channels.

Functional Requirements

Complete list — FR-9 through FR-15 (application experience)

FR-9 Clearly flag predictions made on synthetic (non-real) 13-band data as approximate / unreliable.

FR-10 Display an RGB preview of the uploaded image (true-color bands for real data, RGB for synthetic).

FR-11 Run inference and display the predicted class, its description, and the confidence score.

FR-12 Display the full probability distribution across all 10 classes.

FR-13 Let the user download a synthetic image as a .tif file.

FR-14 Cache loaded models so repeated predictions with the same model don't reload from disk.

FR-15 Handle and report errors gracefully (bad file, wrong band count, missing model) without crashing.

Non-Functional Requirements

Complete list — NFR-1 through NFR-6

NFR-1 — PERFORMANCE
Inference on a single uploaded image completes within a few seconds after model load.

NFR-2 — PERFORMANCE
Switching models doesn't require restarting the app; caching keeps subsequent predictions fast.

NFR-3 — USABILITY
Model selection, upload, preview and results stay in distinct, uncluttered sections.

NFR-4 — USABILITY
Approximation warnings must be visually prominent, never mistaken for a reliable result.

NFR-5 — RELIABILITY
Malformed uploads and load failures are handled gracefully, without crashing the app.

NFR-6 — REPRODUCIBILITY
Training uses a fixed random seed (42) so data splits and results are reproducible.

Non-Functional Requirements

Complete list — NFR-7 through NFR-11

NFR-7 — MAINTAINABILITY

Model configuration lives in a single registry, not scattered across UI code.

NFR-8 — PORTABILITY

Models load from the app's working directory — no hard-coded absolute paths.

NFR-9 — SCALABILITY

Adding a new model requires only a registry entry and a model file, no logic changes.

NFR-10 — COMPATIBILITY

Runs on standard Streamlit hosting with TensorFlow/Keras; no GPU required for inference.

NFR-11 — DATA INTEGRITY

Real vs. synthetic bands stay distinguished end-to-end, including correct RGB band ordering.

Requirements at a Glance

6	9	4	4	15	11
Stakeholder Groups	User Stories	Use Cases Documented	Models Compared	Functional Requirements	Non-Functional Requirements

Derived directly from the EuroSAT training notebook and the Streamlit inference app — reflecting the system as built, plus the stakeholder needs it implies.

Stakeholder Analysis

User Stories & Use Cases

Functional Requirements

Non-Functional Requirements

Thank You